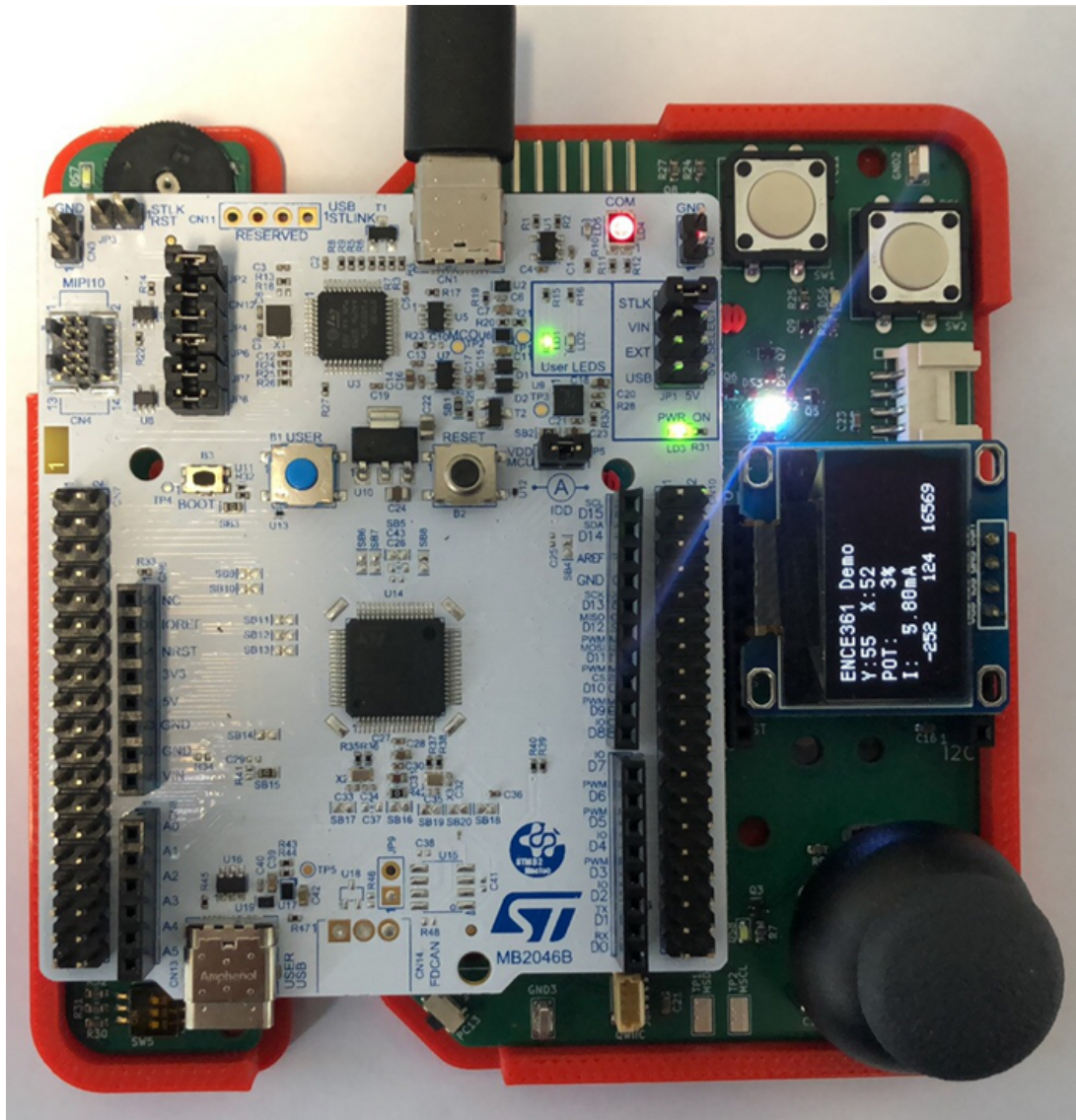


Step Counter Documentation

Curtis Christian, Andy Armour

May 29, 2026



Contents

1	Introduction	3
2	Design Architecture	3
2.1	Module Layering	3
2.2	Modular Functionality	3
2.3	Task Timing	4
2.4	Direct Memory Access	4
2.5	Shared Memory	4
3	Sub-Systems	5
3.1	GUI FSM	5
3.2	IMU Driver	5
3.3	Piezoelectric Buzzer Driver	6
4	Task Scheduler	7
5	Interrupts	8
6	Future Improvements	8

1 Introduction

The aim of this project was to program a STM32C071 board with the RCAP extension to behave as a step counter device. Achieving this goal required the development of an interrupt based kernel with sensible task scheduling. The kernel needed to interface with a number of external peripherals through differing methods and techniques. The interfaced peripherals included: RGB LED's, Input buttons, a Joy-stick, a Piezoelectric buzzer, a Potentiometer, UART communication, and an OLED display. The Step counter can display the current steps, progress to the goal, and distance travelled. The step goal can be adjusted by holding the joystick down for a second on the progress display to allow the goal to be adjusted by the potentiometer. The RGB LEDs show the progress to the goal of 100%, the first increasing in brightness till first 25%. The joystick is used to change display states and units. Testing mode is entered by rapidly double clicking button 2; the joy-stick then alters the states and alters the steps value within a range. Button 4 enables UART communication over a serial connection.

2 Design Architecture

2.1 Module Layering

This structure ensures top down program flow from the application layer to the individual component drivers. The application top module and mostly handles task scheduling, module configuration, and data sharing; it does not have access to the HAL layer. This allows abstraction of features to each module allowing clear intuitive code that is lightly coupled. This light coupling allows for portability of code easily allows for addition of new features without altering and code within modules. This is slightly hampered by the timing module, this is addressed in the future improvements section. This internal module structure attempts to group the most cohesive code together maximally, as shown in Figure 1.

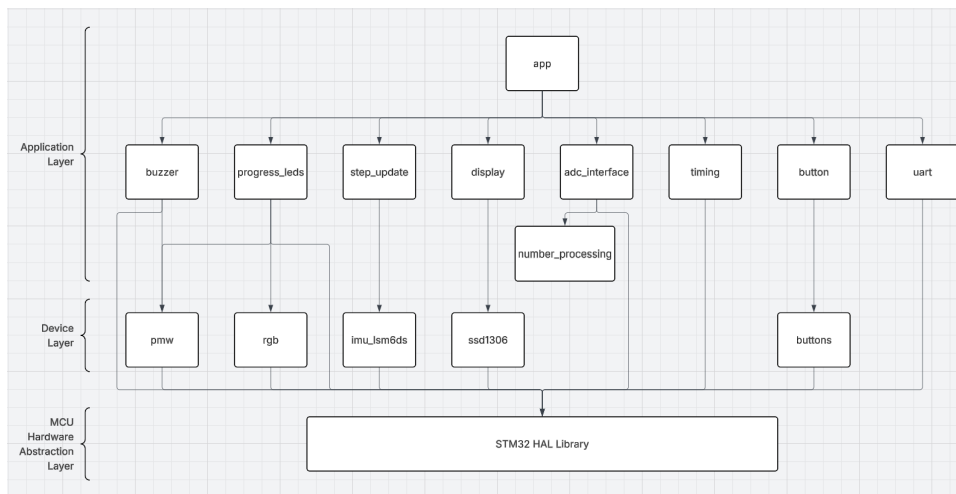


Figure 1: Design Architecture of program

2.2 Modular Functionality

Using the Hardware Abstraction Layer (HAL) library is a key advantage of the architecture of this design. By abstracting low-level operations, the main code is decoupled from the specific micro-controller that is being used. This means that porting the design to a different STM32 device would be significantly easier as the HAL handles the hardware-specific differences internally.

The modularity of the design complements this further by splitting each driver into its own module. Doing so makes debugging and maintenance of the code much easier and more intuitive. If additional functions need to be added, a new module can be coded and tested in isolation before being fully integrated into the design.

2.3 Task Timing

This application runs off of a program tick rate of 1000Hz and poles task refresh rate with frequencies defined in the timing header file. These frequencies are shown in Figure 2.

```
#define BUZZER_TASK_FREQUENCY_HZ 2
#define BUTTON_TASK_FREQUENCY_HZ 100
#define DISPLAY_TASK_FREQUENCY_HZ 6
#define ADC_TASK_FREQUENCY_HZ 10
#define UART_TASK_FREQUENCY_HZ 2
#define LED_TASK_FREQUENCY_HZ 6
```

Figure 2: Task Frequencies Pre-processor Definitions

These frequencies are valid for this case, with button polling occurs at a higher rate as it is relatively low load and requires responsibility. The output tasks occur at lower rate due to cost and low priority level. Some modules of code exhibit the operation avoidance principle. Specifically, the display and the progress LEDs where if there is no change in the values critical to these processes the task will be avoided. This is possible because of the update display and update LEDs flags in the system flags structure. This allows for more efficient and effective use of power and CPU resources.

2.4 Direct Memory Access

The values from the joystick position and the potentiometer were analogue and had to be read through DMA, using block transfer or burst mode. This is required by STM for data reads of more that one data element.

2.5 Shared Memory

In this program the variables are declared as members of related value structures and initialised in the application module. Then these are passed on to the relevant tasks as pointers. This allows for code that performs better due cheaper function calls, while also being more readable. This also is makes the code more portable as values only need to added to the structure definitions and initialisation to be usable through the code. The con to this is that access in not defined and all of the code can access variables that shouldn't. The structures are divided by data type and function, as shown in Figure 3.

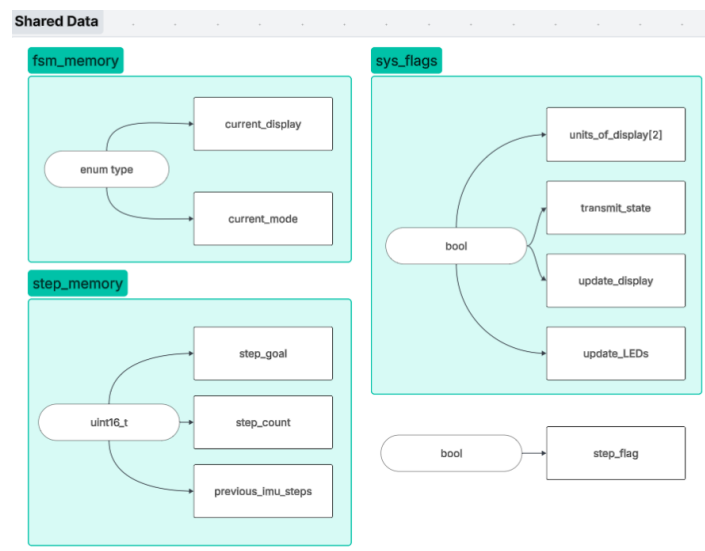


Figure 3: Shared Memory Structure Mapping Diagram

3 Sub-Systems

3.1 GUI FSM

The GUI FSM label definition is in app.h and is set to current steps on boot or reset. The states are current steps, goal progress, and distance travelled. This is defined as an enumeration, and uses a switch in the display module to navigate to the correct label. The states are changed by manipulation with the joystick L and R change state and Within the switch, as shown in Figure 4.

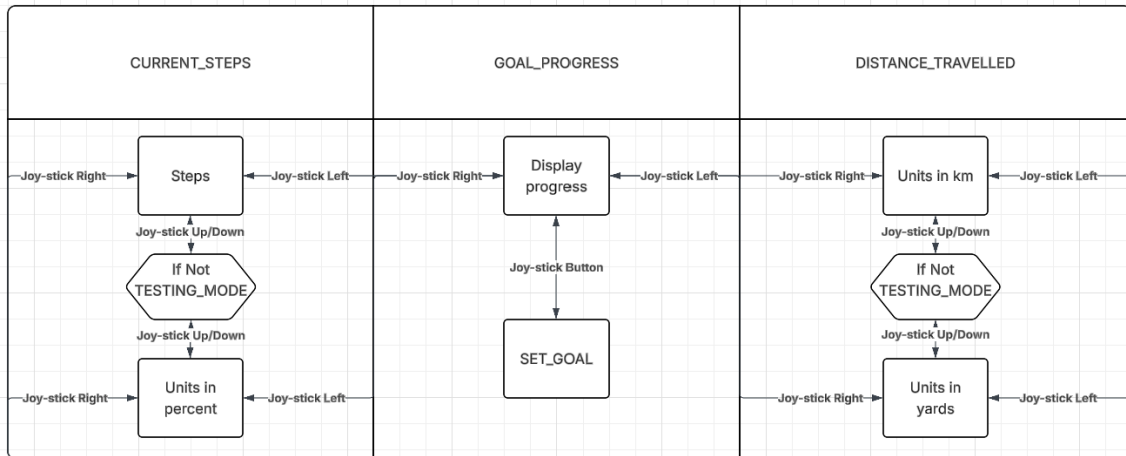


Figure 4: GUI FSM

3.2 IMU Driver

The IMU driver communicates with the LSM6DS IMU over SPI using the `imu_lsm6ds_write_byte()` and `imu_lsm6ds_read_byte()` functions. The step counter task only executes when a rising edge is detected on the INT1 pin.

On each interrupt, `stepUpdateTaskExecute()` reads the 16-bit step count from the `STEP_COUNTER_L` and `STEP_COUNTER_H` registers, calculates the change from the previously stored IMU count, and adds that difference to the running `step_count`, as shown in Figure 5.

NVIC interrupts are briefly disabled during the update to prevent a race condition.

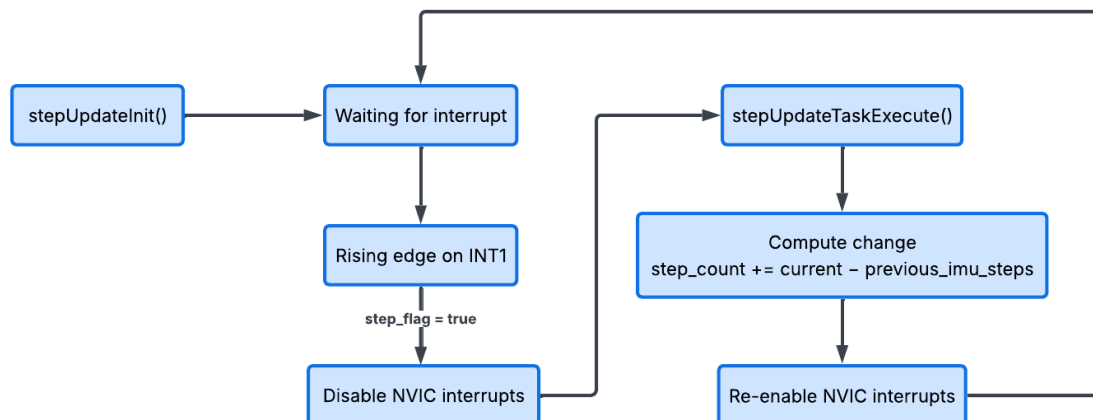


Figure 5: IMU driver and step count flow.

3.3 Piezoelectric Buzzer Driver

The piezoelectric buzzer is primarily driven by the `buzzerSetFrequency()` function. This works by setting TIM16's auto-reload register (ARR) to $\text{TIMER_CLOCK}/\text{freq} - 1$, and the compare value to exactly half the ARR. This sets a 50% duty cycle, ensuring maximum volume.

A small library of note frequencies is defined. Octaves 5 and 6 are currently implemented; more could be easily added to define more complex tunes. Notes are defined as a two-part `Note_t` struct, containing a frequency and a duration. A tune is then defined as an array of `Note_t` structs, terminated by a `NOTE_END`.

Tunes can be played using the `playNote()` and `playTune()` functions. `playNote()` plays the specified frequency for the specified duration using `HAL_Delay()`, and `playTune()` iterates through the note array until it reaches `NOTE_END`.

Two tunes are currently defined, with `tune2` playing on initialisation, and `tune1` playing upon completion of the step goal. Goal completion is managed by `buzzerCheckGoal()`, which uses a static `bool` `victory_latch` to ensure `tune1` fires exactly once when the goal is reached, resetting only if the step count drops back below the goal.

4 Task Scheduler

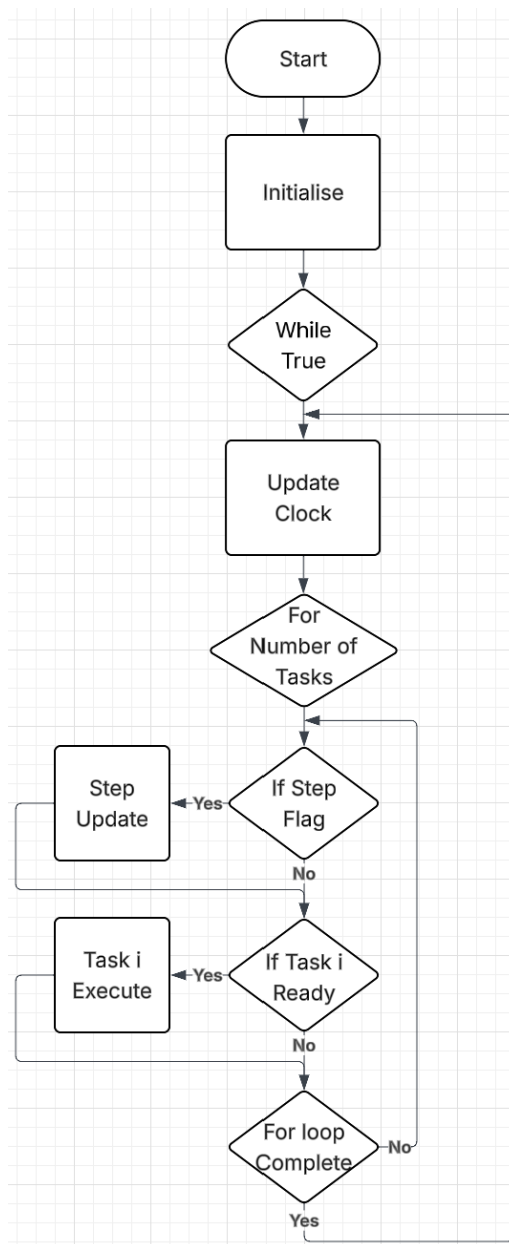


Figure 6: Task Scheduler Diagram

The task scheduler runs by iterating through an enumeration of tasks, where the index of the list of function pointers corresponds to the timing function of that task. It runs the function pointed to, then navigates through a switch via the enumeration labels to run that task function. It then increments the enumeration using the fact it acts as an integer; this progresses through the tasks till the tasks have run out and then loops back to update loop timing.

It essentially acts as a task FSM, checking the interrupt flag each time to minimise code and allow for maximum readability, while also prioritising interrupts maximally.

A weak point of this, as addressed in the future improvements section, is the over reliance on the timing module and lack of automation with creation of a module “object”. Object loosely defines the relation of the Initialisation, Timing, and Method of Execution of a given task. This should also include specification of task frequency internal to the application module to improve portability and lower coupling of modules.

```

while(1)
{
    loopTimer(); // 1000 Hz
    // needs to be here once to prevent race conditions
    for (tasks_t i = DISPLAY; i < NUM_TASKS; i++)
    {
        if (step_flag) // Step interrupt
        {
            stepUpdateTaskExecute(&step_memory, &step_flag);
            sys_flags.update_display = true;
            sys_flags.update_LEDs = true;
        }
        if (taskTiming[i]())
        {
            switch (i)
            {
                case DISPLAY:
                    displayStatusBehaviour(&fsm_memory, &step_memory, &sys_flags);
                    break;
                case UART:
                    if (sys_flags.transmit_state)
                    {
                        uartSerialOutput(&fsm_memory, &step_memory);
                    }
                    break;
                case BUTTONS:
                    buttonInputBehaviour(&fsm_memory, &step_memory, &sys_flags);
                    break;
                case ADC_INTERFACE:
                    joystickADCBehaviour(&fsm_memory, &step_memory, &sys_flags);
                    break;
                case BUZZER:
                    buzzerCheckGoal(&step_memory);
                    break;
                case LEDS:
                    if (sys_flags.update_LEDs)
                    {
                        sys_flags.update_LEDs = false;
                        progressLEDTasksExecute(&step_memory);
                    }
                    break;
            }
        }
    }
}
  
```

Figure 7: Task Scheduler Code

5 Interrupts

This program utilizes an interrupt driven kernel. This means there is a boolean step flag that is set to true when a rising edge is detected from the IMU. In between the complete of each task the program checks this flag and will run the step update task if the flag is raised. In this task to ensure no race conditions, and therefore undefined behaviour occurs, interrupt are disabled when the program goes to update the step data and set the flag to false this is shown in the figure below.

```
void stepUpdateTaskExecute(Step_Values_t* step_memory, bool* step_flag)
{
    HAL_NVIC_DisableIRQ(EXTI4_15_IRQn);
    uint8_t steps_low    = imu_lsm6ds_read_byte(STEP_COUNTER_L);
    uint8_t steps_high   = imu_lsm6ds_read_byte(STEP_COUNTER_H);
    *step_flag           = false;
    HAL_NVIC_EnableIRQ(EXTI4_15_IRQn);
    uint16_t current_imu = (((uint16_t)steps_high) << 8) | steps_low;
    step_memory->step_count += (current_imu - step_memory->previous_imu_steps);
    step_memory->previous_imu_steps = current_imu;
}
```

Figure 8: Update Steps Code Snippet

The Nested Vector Interrupt Controller is used to assign priority to interrupts. This functionality isn't fully utilised by our application as only one programmed interrupt is directly used.

6 Future Improvements

Breaking the display task and the buzzer task into partial completed task FSM states to allow for multitasking (small chunks of tasks at a time, for larger tasks). An example diagram and an example of the code template of this multi-tasking concept is shown in the figures below.

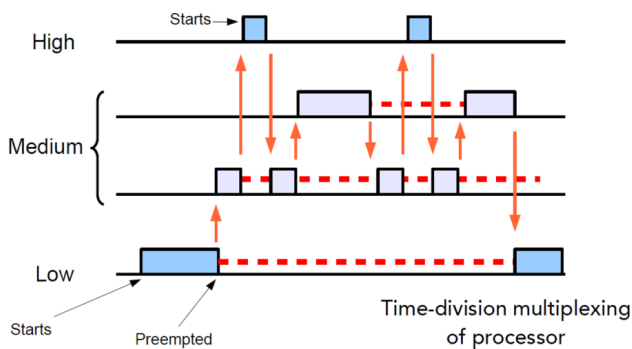


Figure 9: Multi-tasking Diagram

```
void TaskA() {
    static TaskAState_t state = START;
    switch (state)
    {
        case START:
            // Do some stuff
            state = STATE_1;
            break;

        case STATE_1:
            // Do some more stuff
            state = STATE_2;
            break;
        ...
    }
    return;
}
```

Figure 10: Multi-tasking Example

Encapsulation Structure to reduce further pointers passed through function calls. Further optimisation of the function pointers with attributes of the task form becoming object like, with sub-tasks having some form of inheritance. This should allow for a structure of code that with features added or removed does require much change to the application layer. Timer objects and specified frequencies brought back to the application layer with the ability to generate timer functionality without modification to the Timer module. The timer module is very bad as it isn't automated. Removal of the number processing module and refinement of that code. Read and write modular specifications of data would ensure unintended writes don't occur while still allowing for scalability with additional features.